

RENOMEAÇÃO E ORGANIZAÇÃO DE DADOS

Unifor

Resposta — Questão da Aula 8



Dado o DataFrame `df` com as colunas 'cargo' e 'salario', qual código calcula a média salarial para cada cargo usando `groupby()`?

A

```
df['salario'].groupby('cargo').mean()
```

B

```
df.groupby('cargo')['salario'].mean()
```

C

```
df.groupby(['salario'])['cargo'].mean()
```

D

```
df['cargo'].mean().groupby('salario')
```

Resposta: B — Em Pandas, usamos `df.groupby('coluna_grupo')['coluna_valor'].funcao()` para agrupar e agregar.

Objetivos da Aula

Renomear Colunas

Usar o `.rename()` para padronizar os nomes das colunas de um DataFrame.

Reordenar Colunas

Selecionar e reorganizar colunas em uma nova ordem usando listas.

Colunas Derivadas

Criar novas colunas calculadas a partir de colunas existentes.

Renomear o Índice

Usar `.rename()` também no índice das linhas quando necessário.

Boas Práticas

Padronizar nomes: minúsculas, sem espaços, com underscores.

Parte Prática

Aplicar tudo em um dataset real, organizando e enriquecendo os dados.

Renomeando Colunas com .rename()

Sintaxe básica:

```
df.rename(columns={'nome_antigo':  
                  'nome_novo'})
```

Renomear múltiplas colunas:

```
df.rename(columns={  
    'nome': 'Nome',  
    'idade': 'Idade',  
    'cidade': 'Cidade'  
})
```

Com inplace=True (modifica direto):

```
df.rename(columns={'nome': 'Nome'},  
          inplace=True)
```

Exemplos práticos:

Remover espaços do nome

```
df.rename(columns={'nome completo':  
                  'nome_completo'})
```

Padronizar para minúsculas

```
df.columns = df.columns.str.lower()  
df.columns = df.columns.str.replace(' ', '_')
```

Renomear pelo índice

```
df.rename(index={0: 'primeiro',  
                1: 'segundo'})
```

Reordenando e Selecionando Colunas

Selecionar colunas:

Uma coluna (Series)

```
df['nome']
```

Múltiplas colunas (DataFrame)

```
df[['nome', 'idade', 'salario']]
```

Reordenar todas as colunas

```
nova_ordem = ['cidade', 'nome', 'salario']  
df = df[nova_ordem]
```

Remover colunas com .drop():

```
# Remover uma coluna  
df.drop(columns=['coluna'])  
  
# Remover múltiplas  
df.drop(columns=['col1', 'col2'])  
  
# Com inplace=True  
df.drop(columns=['col1'], inplace=True)
```

Dica: use `df.columns` para ver todos os nomes das colunas antes de reordenar ou remover.

Criando Colunas Derivadas

O que são?

→ Novas colunas calculadas a partir de colunas já existentes.

→ Permitem enriquecer o dataset sem alterar os dados originais.

→ Criadas com uma simples atribuição: `df['nova'] = ...`

Sintaxe:

```
df['nova_coluna'] = operacao_com_colunas
```

Exemplos práticos:

Salário anual

```
df['salario_anual'] = df['salario'] * 12
```

Nome completo

```
df['nome_completo'] = df['nome'] + ' ' +  
df['sobrenome']
```

Classificação por salário

```
df['nivel'] = df['salario'].apply(  
    lambda x: 'Alto' if x > 5000 else  
    'Baixo')
```

Diferença entre colunas

```
df['bonus'] = df['salario'] *  
df['perc_bonus']
```

Parte Prática — Organizando um Dataset

```
import pandas as pd

df = pd.read_csv('funcionarios.csv')

# Ver nomes das colunas atuais
print(df.columns.tolist())

# Renomear colunas
df.rename(columns={
    'nome': 'Nome',
    'idade': 'Idade',
    'cidade': 'Cidade',
    'salario': 'Salario'
}, inplace=True)

# Reordenar colunas
df = df[['Nome', 'Cidade', 'Idade', 'Salario']]

# Criar coluna derivada
df['Salario_Anual'] = df['Salario'] * 12

# Remover coluna desnecessária
df.drop(columns=['Idade'], inplace=True)
```

Questão — Aula 9



Qual código renomeia corretamente a coluna 'salario' para 'Salario' em um DataFrame df, modificando-o diretamente (sem criar um novo)?

A

```
df.columns['salario'] = 'Salario'
```

B

```
df.rename('salario', 'Salario', inplace=True)
```

C

```
df.rename(columns={'salario': 'Salario'}, inplace=True)
```

D

```
df['salario'].rename('Salario')
```

Resposta: C — `.rename()` exige o argumento `columns={}` com o mapeamento, e `inplace=True` para modificar direto.